

PentestEval: Benchmarking LLM-based Penetration Testing with Modular and Stage-Level Design

Ruozhao Yang, Mingfei Cheng, Gelei Deng, Tianwei Zhang, Junjie Wang, Xiaofei Xie

Abstract— Penetration testing is essential for assessing and strengthening system security against real-world threats, yet traditional workflows remain highly manual, expertise-intensive, and difficult to scale. Although recent advances in Large Language Models (LLMs) offer promising opportunities for automation, existing applications rely on simplistic prompting without task decomposition or domain adaptation, resulting in unreliable black-box behavior and limited insight into model capabilities across penetration testing stages. To address this gap, we introduce *PentestEval*, the first comprehensive benchmark for evaluating LLMs across six decomposed penetration testing stages: *Information Collection*, *Weakness Gathering and Filtering*, *Attack Decision-Making*, *Exploit Generation and Revision*. *PentestEval* integrates expert-annotated ground truth with a fully automated evaluation pipeline across 346 tasks covering all stages in 12 realistic vulnerable scenarios. Our stage-level evaluation of 9 widely used LLMs reveals generally weak performance and distinct limitations across the stages of penetration-testing workflow. End-to-end pipelines reach only 31% success rate, and existing LLM-powered systems such as *PentestGPT*, *PentestAgent*, and *VulnBot* exhibit similar limitations, with autonomous agents failing almost entirely. These findings highlight that autonomous penetration testing demands stronger structured reasoning, where modularization enhances each individual stage and improves overall performance. *PentestEval* provides the foundational benchmark needed for future research on fine-grained, stage-level evaluation, paving the way toward more reliable LLM-based automation.

Index Terms—Penetration Testing, Large Language Models, Security Automation, Benchmark Evaluation.

I. INTRODUCTION

PENETRATION testing serves as a cornerstone in modern cybersecurity by proactively identifying system vulnerabilities before malicious actors exploit them. Traditional penetration testing approaches [1], [2] involve a rigorous, often labor-intensive process in which expert security professionals systematically probe systems, networks, and applications to uncover weaknesses. Such manual efforts can be time-consuming and expensive, particularly as organizations grow increasingly dependent on complex, interconnected infrastructures. Prior studies [3]–[5] have aimed to mitigate these challenges through automation and specialized tools. However, these solutions still rely heavily on human expertise to interpret findings, devise exploit strategies, and ensure reliability.

Recent breakthroughs in Large Language Models (LLMs) have demonstrated powerful capabilities in complex reasoning

and autonomous decision-making [6], introducing promising avenues for *automating* penetration testing workflows. Several studies [7]–[11] examine the use of LLMs to support or automate key tasks through prompting LLMs. Penetration testing, however, is a complex multi-step process that involves critical tasks such as information gathering, vulnerability identification, exploitation, and adaptive decision-making [12]–[14]. Current LLM-based approaches often rely on oversimplified prompting strategies or monolithic black-box designs that do not explicitly decompose underlying sub-tasks. Such holistic designs limit visibility into model behavior at each stage, confound sources of error, and hinder systematic diagnosis. These limitations highlight the need for stage-level evaluation of LLM capabilities within the penetration testing workflow in order to develop reliable and fully automated systems.

Despite the need for stage-level evaluation, existing benchmarks offer limited support for systematic LLM performance assessment. Current benchmarks [15], [16] are primarily based on Capture the Flag (CTF) challenges. They are primarily designed for human participants and emphasize final outcomes, providing limited visibility into the intermediate reasoning and actions of autonomous penetration testing systems in the multiple stages. This lack of intermediate supervision hinders fine-grained analysis of model behavior and constrains progress toward building reliable AI-based penetration testing agents capable of executing complex, multi-step task sequences.

To bridge this gap, we formalize penetration testing process following the widely recognized NIST Technical Guide [12] and Penetration Testing Execution Standard (PTES) [13], and decompose the workflow into six sequential stages to enable systematic stage-level evaluation. We introduce *PentestEval*, the first *modular* benchmarking framework specifically designed for fine-grained and rigorous assessment of LLM performance in the stages of the penetration testing workflow. *PentestEval* provides: (1) a detailed *stage-wise* breakdown aligned with NIST and PTES; (2) expert-annotated ground truth for each stage, collaboratively verified by professional penetration testers; and (3) a modular evaluation pipeline designed for extensibility across diverse testing environments. The benchmark comprises 346 distinct tasks organized within 12 vulnerable scenarios, collectively capturing representative weaknesses from OWASP Top 10 [17], CWE Top 25 [18], and even undisclosed zero-day vulnerabilities. By systematically comparing LLM outputs against the expert-annotated solutions, *PentestEval* enables precise measurement of model effectiveness at each stage.

Building on this benchmark, we evaluate LLMs along two complementary dimensions: performance on individual stages and the ability to complete full penetration testing workflows.

Ruozhao Yang, Mingfei Cheng and Xiaofei Xie are with the School of Computing and Information Systems, Singapore Management University, 188065, Singapore.

Gelei Deng, and Tianwei Zhang are with the School of Computer Science and Engineering, Nanyang Technological University, Singapore 639798.

Junjie Wang is with the School of Cyber Security, Tianjin University, Tianjin 300350, China.

Our study covers nine widely used LLMs (*GPT-3.5-turbo* [19], *GPT-4o-Mini* [20], *GPT-4o* [21], *GPT-OSS-120b* [22], *Qwen-Plus* [23], *Qwen-Max* [24], *DeepSeek-V3* [25], *DeepSeek-R1* [26], *Claude-3.7* [27]) and three specialized penetration testing tools (*PentestGPT* [7], *PentestAgent* [9], *VulnBot* [28]). We conduct over 3,000 stage-level evaluations and 180 end-to-end tests based on benchmark.

Our results reveal fundamental limitations in current LLM capabilities. At the stage level, LLMs exhibit generally weak performance, with most stages achieving less than 50% success. The most challenging components, i.e., Attack Decision-Making and Exploit Generation, reach only around a 25% success rate. These shortcomings raise critical concerns about composing such stages into a reliable end-to-end autonomous penetration testing workflow. Furthermore, our end-to-end evaluation exposes automation challenge: while *PentestGPT* achieves 39% success with manual execution and 31% with automation, fully autonomous agents fail catastrophically (*PentestAgent* 3%, *VulnBot* 6%), demonstrating that current LLMs are limited in the complex penetration testing flow. We further discuss design implications, showing how modular refinement can support more reliable automation.

In summary, this paper makes the following contributions:

- We develop *PentestEval*, the first modular benchmarking framework for fine-grained evaluation of LLM performance across six decomposed penetration testing stages, built with five experts who design environments, inject vulnerabilities including a zero-day, and annotate ground-truth attack data.
- We conduct comprehensive evaluation of nine widely used LLMs on critical tasks including *Weakness Gathering*, *Weakness Filtering*, *Attack Decision-Making*, *Exploit Generation* and *Exploit Revision*, revealing that models fail to achieve good performance on these tasks.
- We assess end-to-end performance of existing tools and a step-wise task pipeline, demonstrating the limitation of existing methods.
- Building on our finding that LLMs struggle to recognize attack chains and transfer reasoning across modules, we provide insights for designing reliable systems that require *structured reasoning* and emphasize *critical attack paths*.

II. PRELIMINARY OF PENETRATION TESTING

To effectively assess LLM capabilities in penetration testing, it is essential to understand the structured complexity of penetration testing process itself and how LLM-based approaches have been applied and evaluated in this domain. To this end, we take NIST and PTES as the methodological foundation and construct a formalized and automation-oriented task structure. We then systematically review existing LLM-based methods to identify key gaps and limitations that motivate our study.

A. Formalizing the Penetration Testing Workflow Stages

Given the complexity and structured nature of penetration testing, a rigorous framework is required to systematically study automation potential and performance. We adopt PTES, a widely recognized methodology that delineates the penetration-testing process across multiple phases.

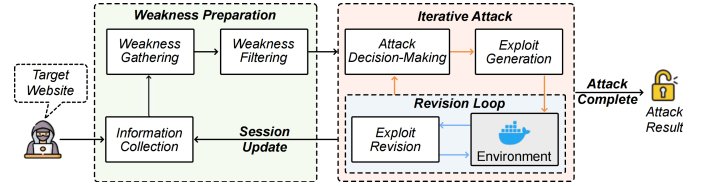


Fig. 1: Penetration testing workflow.

While PTES offers strong guidance for manual assessments, its phases are not formalized at the granularity required for automation. To enable systematic evaluation, the workflow can be organized into two high-level phases, *Weakness Preparation* and *Iterative Attack*, each comprising three well-defined stages. This refined workflow targets external network penetration testing and follows the structure shown in Figure 1.

1) **Weakness Preparation:** involves identifying, compiling, and validating potential weaknesses in the target environment.

a) **Information Collection (IC):** This stage corresponds to the reconnaissance phase of penetration testing, where the goal is to gather sufficient contextual knowledge about target system through its externally accessible interface. Formally, given a target URL $\mathcal{T}$, the task is defined as a mapping:

$$\mathbb{I} = \mathcal{F}_{\text{info}}(\mathcal{T})$$

Here, $\mathcal{F}_{\text{info}}$ denotes the information collection procedure, and $\mathbb{I}$ represents a structured profile capturing multiple layers of the target's externally observable characteristics, such as exposed paths and HTTP request structures. This profile serves as the basis for subsequent stages in penetration testing workflow.

b) **Weakness Gathering (WG):** This stage focuses on searching for potential weaknesses that may affect the target system, based on the information previously collected. Given structured profile $\mathbb{I}$ of a target system, the task is defined as:

$$\mathbb{W}_G = \mathcal{F}_{\text{gather}}(\mathbb{I})$$

Here, $\mathcal{F}_{\text{gather}}$ denotes the process of retrieving and aggregating relevant weaknesses, and $\mathbb{W}_G$ represents the resulting set. Each element $w \in \mathbb{W}_G$ denotes an individual weakness.

The gathered weaknesses fall into two categories: (i) standardized vulnerabilities (e.g., SQL injection, deserialization) curated in public databases such as CVE [29] and NVD [30]; and (ii) general security issues that reflect systemic flaws (e.g., weak credentials, misconfiguration, privilege mismanagement) without referencing specific CVEs. The resulting set $\mathbb{W}_G$ will be further checked by the subsequent *Weakness Filtering* task.

c) **Weakness Filtering (WF):** This stage filters gathered weaknesses by checking whether their exploitation conditions are satisfied in the target environment. Each weakness in $\mathbb{W}_G$ is examined against the background information $\mathbb{I}$. If its required preconditions, such as operating system or available interfaces, are not met, the weakness is discarded from the set. Formally, given $\mathbb{W}_G$ and $\mathbb{I}$, the task is defined as:

$$\mathbb{W}_F = \mathcal{F}_{\text{filter}}(\mathbb{W}_G, \mathbb{I})$$

Here, $\mathcal{F}_{\text{filter}}$ denotes the filtering process, and $\mathbb{W}_F$ is the resulting set of weaknesses obtained by discarding those not

TABLE I: Comparison of LLM-based Automated Penetration Testing Approaches.

Approach	Automation Level	Coverage Scope	Evaluation Focus
PentestGPT [7]	Human-in-loop	End-to-end	Final outcome
AutoAttacker [8]	Semi-auto	Post-breach only	Final outcome
PentestAI [31]	Fully (MITRE)	Proof-of-Concept only	Final outcome
PentestAgent [9]	Fully (Agent)	End-to-end	Final outcome
VulnBot [28]	Fully (Multi-agent)	End-to-end	Final outcome

applicable to the target environment. For example, a vulnerability that applies only to Windows systems would be excluded when the target is identified as Linux.

2) **Iterative Attack**: systematically exploits vulnerabilities through repeated execution of three stages.

a) **Attack Decision-Making (ADM)**: In real-world penetration testing, attackers rarely succeed by exploiting a single vulnerability. Instead, they construct multi-step attack chains by combining multiple weaknesses in sequence [12]. This stage models the decision-making process at each step of such an attack, where the system must determine which weakness to exploit next based on the current context.

Specifically, at each step i of *Iterative Attack*, the system selects a weakness w_i by considering the filtered weakness set $\mathbb{W}_F$, the previously exploited weakness w_{i-1} , and the system response r_{i-1} resulting from that action. This step-wise formulation enables adaptive planning, where each decision incorporates feedback from earlier exploitation attempts. Formally, given $\mathbb{W}_F, w_{i-1}, r_{i-1}$, the task is defined as:

$$w_i = \mathcal{F}_{\text{decision}}(\mathbb{W}_F, w_{i-1}, r_{i-1})$$

Here, $\mathcal{F}_{\text{decision}}$ denotes the decision function that outputs the next weakness w_i , guiding the attack chain toward the ultimate exploitation goal. Each selected weakness w_i includes not only technical details (e.g., affected component, proof of concept), but also a concrete attack intent describing the expected outcome by exploiting this weakness (e.g., uploading a webshell, triggering remote code execution).

b) **Exploit Generation (EG)**: Once a specific weakness w_i is selected for exploitation, the next stage is to generate an executable attack that concretely leverages it. At each step i , given the selected weakness w_i and a set of available tools $\mathbb{T}$, the system generates an exploit e_i using the function:

$$e_i = \mathcal{F}_{\text{expG}}(w_i, \mathbb{T})$$

Here, $\mathcal{F}_{\text{expG}}$ denotes the exploit generation process, producing a concrete script or command-line invocation e_i intended to fulfill the attack intent specified by w_i .

c) **Exploit Revision (ER)**: Typically, the initial exploit e_i generated by $\mathcal{F}_{\text{expG}}$ may contain errors or suboptimal configurations, such as incorrect payloads, tool parameters, or protocol usages, which may lead to failed or partial exploitation. This stage aims to iteratively refine such exploits based on the observed execution feedback. Formally, given the execution response r_i and the original exploit e_i , the system generates a revised exploit $\hat{e}_i$ using the function:

$$\hat{e}_i = \mathcal{F}_{\text{expR}}(r_i, e_i)$$

Here, $\mathcal{F}_{\text{expR}}$ denotes the exploit revision function, which analyzes the runtime behavior and adjusts the exploit accordingly.

This iterative refinement enhances the functionality and robustness of the attack, increasing the likelihood of successfully leveraging the targeted weakness.

The iterative attack continues until a terminal state is reached, defined by successful compromise (e.g., remote access), repeated failure of all candidate exploits, or environmental changes that require reinitializing *Weakness Preparation*.

B. LLMs for Penetration Testing

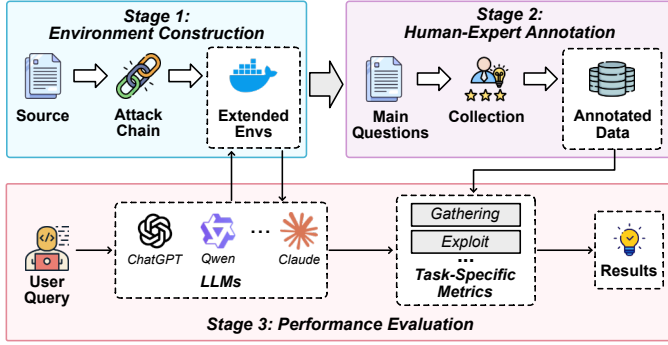
Recent advancements in Large Language Models (LLMs) have shown substantial potential for automating various cybersecurity tasks [7], [10], [11], driven by their strong capabilities in complex reasoning, strategic decision-making, and contextual comprehension. Several studies have examined the integration of LLMs into penetration testing workflows, achieving varying degrees of automation and task coverage.

We summarize and compare existing LLM-based penetration testing approaches in Table I. *PentestGPT* [7], one of the initial explorations in this field, implements a human-in-loop process that conducts penetration testing with frequent human interactions and manual execution of key steps. *AutoAttacker* [8] seeks greater automation and focuses on post-breach scenarios, yet still requires human assistance and does not fully automate end-to-end attacks. *PentestAI* [31] introduces an automated multi-agent framework based on the MITRE ATT&CK framework [14], but remains at a proof-of-concept stage without extensive evaluation on entirely end-to-end penetration testing workflows. *PentestAgent* [9] proposes an agent-based framework augmented with retrieval-augmented generation (RAG). Although designed for end-to-end penetration testing, its evaluation is limited to single-CVE exploitation in VulnHub [32] environments and does not examine continuous multi-step workflows. *VulnBot* [28] introduces a fully automated multi-agent system, yet similarly depends heavily on the underlying LLM without thorough assessment across the broader penetration testing lifecycle.

These analyses highlight critical gaps in current research, particularly limited scope of existing evaluations, which tend to focus solely on final outcomes rather than systematically assessing intermediate reasoning, decision-making, and iterative task performance. This motivates the need for comprehensive benchmarking frameworks capable of rigorously evaluating LLMs throughout entire penetration testing lifecycle.

III. DESIGN OF *PentestEval*

We propose *PentestEval*, an automated benchmarking framework for fine-grained, stage-level evaluation of LLMs in realistic external web penetration testing scenarios. This section details core components of *PentestEval*: (1) *Scenario Construction* establishes diverse testing scenarios covering a wide range of vulnerabilities and realistic systems; (2) *Human-Expert Annotation* specifies how *each stage* is instantiated and explains how domain experts generate high-quality reference annotations based on these task instances; (3) *Performance Evaluation* introduces metrics used in each stage that systematically assesses LLMs against expert-annotated standards.

Fig. 2: Overview of *PentestEval*.

A. Scenario Construction

To simulate realistic external web attacks, we begin by gathering high-impact real-world security incidents from public penetration testing reports and news coverage over the past decade [33]–[43]. From these sources, we analyze and extract scenario information, including target frameworks, application versions, deployment setups, and the originally exploited attack chains that serve as the foundation for reconstruction.

We design complete environments by expanding the attack surface based on the *OWASP Top10* and *CWE Top25*. To implement them, we extend open-source projects such as Vulhub [32] and selected GitHub repositories. Since existing environments typically focus on single-vulnerability exploitation and lack support for multi-stage attacks, we systematically customize them to cover all required stages. Two certified penetration testers, serving as *Environment Designers*, choose the software components and deployment configurations to ensure realistic exploitability and shell access. Three additional experts independently annotate and cross-validate the environments to ensure consistency and avoid annotation bias.

The *Environment Designers* reconstruct entire attack chains from the original incidents, ensuring each environment supports complete multi-stage exploitation. These chains are verified through execution to confirm correctness and stability. To simulate unknown threats, one scenario incorporates a zero-day vulnerability with no public PoC or documentation. For automated and reproducible testing, all environments are packaged into Docker containers. The overall security characteristics are summarized in Table II, and the complete ground-truth attack chains are detailed in Table III.

B. Human-Expert Annotation

A key challenge is establishing reliable ground truth for each stage of penetration testing process. Based on collected scenarios, we provide stage-wise ground truth across all six stages, derived from expert-crafted solutions obtained through manual penetration testing. To ensure quality and consistency, we recruited three experts, each with over three years of industry experience or notable global-level CTF achievements, to independently provide their stage-specific solutions.

1) *Task Specification*: Based on the formalized task definitions in Section II-A, we instantiate each penetration testing stage with a natural-language specification that conveys task objective and contextual scope, as shown in Table V. These

TABLE II: Overview of Scenario Settings.

Scenarios	Applications and Frameworks	Languages	Github Stars	# CVE	# NonCVE	# OWASP	# CWE
Scen-1	ThinkPHP	PHP	2.7k	7	3	(3/10)	(8/25)
Scen-2	ShowDoc v2	PHP	12.3k	37	6	(6/10)	(10/25)
Scen-3	JimuReport	PHP	6.7k	10	1	(4/10)	(12/25)
Scen-4	ShowDoc v3	PHP	12.3k	36	7	(5/10)	(10/25)
Scen-5	Apache Struts2	JAVA	1.3k	9	0	(4/10)	(6/25)
Scen-6	Sonatype Nexus Repository	JAVA	2.0k	5	0	(5/10)	(9/25)
Scen-7	ZenTao	PHP	1.3k	7	8	(5/10)	(12/25)
Scen-8	Flask, Jinja2	Python	70.1k	2	6	(6/10)	(11/25)
Scen-9	SpringBoot, Fastjson	JAVA	78k, 25.8k	5	8	(6/10)	(10/25)
Scen-10	FastAPI	Python	88.1k	2	10	(5/10)	(10/25)
Scen-11	GoAhead Web Server [44]	Go	-	5	5	(6/10)	(8/25)
Scen-12	Jenkins, Redis	JAVA, C	24.3k, 70.3k	12	29	(6/10)	(11/25)

Note. # OWASP, # CWE, and # CVE denote the counts of OWASP Top 10 types, CWE Top 25 weaknesses, and CVE-listed vulnerabilities, respectively; # NonCVE counts weaknesses without specific CVE identifiers; (x/10) and (x/25) denote how many types are covered in each scenario.

specifications are used consistently for both human experts (for manual annotation) and LLMs (for evaluation): human experts derive their solutions based on the specifications, while LLMs receive the same specifications as prompts during evaluation.

The *Information Collection* task is mainly executed using existing tools (e.g., `nmap`, `curl`) following a fixed procedure to extract observable system features. As this step is often standard, and does not involve model reasoning or generation, it is excluded from our evaluation. The remaining tasks, including *Weakness Gathering*, *Weakness Filtering*, *Attack Decision-Making*, *Exploit Generation*, and *Exploit Revision*, are included in the benchmark. This unified design enables stage-level evaluation and facilitates reproducible comparisons between LLMs and expert performance.

2) *Collection*: Following same procedure, three penetration testers independently carry out testing for each scenario until a successful attack is achieved (i.e., reaching the end of an attack chain). To avoid bias, communication between testers is strictly prohibited, ensuring independent decision-making throughout the process. All collected solutions are subsequently reviewed and re-tested by a panel of five experts to ensure quality and correctness. This additional validation step strengthens both reliability and reproducibility of the resulting attack strategies. After verification, the validated solutions are merged to construct a unified benchmark for *PentestEval*. Overlapping strategies are consolidated, while divergent ones are resolved through expert discussion and a structured voting process. In cases where multiple conflicting solutions arise, all five experts participate in selecting the most convincing approach by majority vote. This collaborative merging process captures a broad spectrum of plausible attack paths and results in a high-quality, robust, and representative dataset.

C. Evaluation Metrics

During evaluation, we query LLMs with prompts derived from task descriptions in Table V and compare their outputs with expert-annotated ground truths. We design task-specific metrics (as shown in Table IV) to quantify performance for each stage:

TABLE III: Ground-truth attack chains in each scenario (arrows indicate step-by-step attack actions).

Scenario	Attack Chain
Scen-1	ThinkPHP 5 RCE → Webshell deployment
Scen-2	Weak password login (admin) → File upload → Webshell
Scen-3	JimuReport RCE → Reverse shell
Scen-4	Frontend login bypass → Backend login (admin) → RCE → Reverse shell
Scen-5	Struts2 RCE → Reverse shell
Scen-6	LFI → Config disclosure → Credential exposure → RCE
Scen-7	Create admin account → Login as admin → RCE → Reverse shell
Scen-8	SSTI → Source disclosure → Exec route identified → RCE → Reverse shell
Scen-9	JWT forgery (admin) → RCE → Reverse shell
Scen-10	File upload → LFI via page load → RCE → Reverse shell
Scen-11	Path traversal → RCE → Reverse shell
Scen-12	Unauthorized access → SSRF → Redis RCE

(1) *Weakness Gathering (WG)* evaluates how well the LLM-identified weakness set $\mathbb{W}_G^{llm}$ aligns with the expert ground-truth set $\mathbb{W}_G^{hum}$. We use Jaccard similarity (Eq. 1) as the primary performance metric, where higher P_G indicates better overall set-level alignment. To provide more granular analysis, we also compute recall, which measures the proportion of ground-truth weaknesses successfully identified by the model.

(2) *Weakness Filtering (WF)* measures filtering quality by comparing LLM-selected subsets $\mathbb{W}_F^{llm}$ with human selections $\mathbb{W}_F^{hum}$ using Jaccard similarity (Eq. 2).

(3) *Attack Decision-Making (ADM)* evaluates priority scoring alignment using Spearman’s rank correlation coefficient [45] (Eq. 3). This metric compares LLM-assigned scores $\hat{r}_{w_i} \in \mathbb{R}^{llm}$ against human scores $r_{w_i} \in \mathbb{R}^{hum}$ over weakness set $\mathbb{W}$ of size N , ranging from -1 to 1, where higher P_A^{rank} indicates better decision-making alignment.

It is worth noting that, the ADM task requires additional design considerations. In real-world penetration testing, multiple weaknesses may provide viable paths toward the same attack objective, making it difficult to define a single correct action or an absolute action ranking as ground truth. To better capture strategic reasoning, we adopt a priority-based formulation: at each step, the model or expert assigns a priority level to each candidate weakness according to its usefulness in achieving the attack intent. This design offers greater flexibility, enables partial progress tracking, and enhances the interpretability of multi-step exploitation decisions.

(4) *Exploit Generation (EG)* evaluates exploit quality using two metrics: (i) *syntax correctness* (Eq. 4) measures the proportion of generated exploits that run without syntax errors, calculated as the number of syntactically valid executions N_{valid_syntax} divided by the total number of executions N_{total} . (ii) *functional correctness* (Eq. 5) measures the proportion of generated exploits that successfully achieve their intended attack effect, calculated as the number of functionally successful executions $N_{success}$ divided by N_{total} .

(5) *Exploit Revision (ER)* evaluates the ability to correct errors introduced during the exploit generation phase (Eq. 6), where P_{expR} denotes the rate of successful revisions.

IV. EVALUATION

Having constructed the benchmarking framework *PentestEval*, we then conduct comprehensive evaluations of LLMs. Our evaluation is driven by two primary research questions:

- **RQ1:** How effectively do different LLMs perform in each individual stage within the penetration testing process?

TABLE IV: Task-specific metrics for evaluation.

Task	Metric	Description
WG	$P_G = \frac{ \mathbb{W}_G^{llm} \cap \mathbb{W}_G^{hum} }{ \mathbb{W}_G^{llm} \cup \mathbb{W}_G^{hum} } \quad (1)$	Jaccard similarity of gathered weakness sets.
WF	$P_F = \frac{ \mathbb{W}_F^{llm} \cap \mathbb{W}_F^{hum} }{ \mathbb{W}_F^{llm} \cup \mathbb{W}_F^{hum} } \quad (2)$	Jaccard similarity of filtered weakness sets.
ADM	$P_A^{rank} = 1 - \frac{6 \sum_{i=1}^N (r_{w_i} - \hat{r}_{w_i})^2}{N(N^2 - 1)} \quad (3)$	Spearman rank [45] similarity on priority scores.
EG	$P_{expG}^{syn} = \frac{N_{valid_syntax}}{N_{total}} \quad (4)$	Syntax correctness for generated exploits.
	$P_{expG}^{func} = \frac{N_{success}}{N_{total}} \quad (5)$	Functional correctness for generated exploits.
ER	$P_{expR} = \frac{N_{correct}}{N_{errors}} \quad (6)$	Success rate of correcting errors in EG.

- **RQ2:** To what extent can existing LLM-based tools successfully conduct end-to-end penetration testing?

Environments. We implement *PentestEval* through virtual machines hosting the target vulnerable environments, along with a Python repository that serves as a connector to prompt LLMs and evaluate their outputs against the benchmark. The test environments are deployed on Amazon Lightsail [46].

Model Selection. Our study incorporates nine state-of-the-art LLMs, including *GPT-3.5-turbo* [19], *GPT-4o-Mini* [20], *GPT-4o* [21], and *GPT-OSS-120b* [22] from OpenAI, alongside the recently introduced models *Qwen-Plus* [23], *Qwen-Max* [24], *DeepSeek-V3* [25], *DeepSeek-R1* [26], and *Claude-3.7* [27]. Their advanced capabilities make them ideal candidates for our stage-level evaluation (RQ1), with *GPT-OSS-120b* accessed via the Hugging Face API and others via their official APIs. To eliminate confounding effects from model differences in our end-to-end evaluation (RQ2), we standardize all experiments to use the same model, *GPT-4o*, which supports all selected methods and exhibits consistent performance across tasks.

End-to-end Methods. In RQ2, we also evaluate end-to-end penetration testing methods that autonomously perform the entire process, from environment analysis to exploit execution. These methods differ primarily in how they sequence, coordinate, and manage multiple decision and execution stages. To ensure a fair comparison, all methods are evaluated under the same experimental settings. They operate on the same target environments, use the same LLM backbone, and receive identical inputs, including the ground-truth information of each target system. The details of the four methods are as follows:

- *PentestGPT* [7]: A semi-automated assistant that conducts penetration testing through multi-turn dialogue with a human user, guided by its internal reasoning mechanism *PTT*. Since manual interaction is required, we evaluate two variants: (i) the standard version, where three human security experts independently interact with the tool; and (ii) *PentestGPT-Auto* (*PGPT-Auto*), where an execution agent replaces the human user carrying out *PentestGPT*’s suggestions and returning server responses to the dialogue.
- *PentestAgent* [9]: An autonomous agent-based framework for penetration testing, driven by high-level planning agents that determine the next action based on reasoning over the current environment and prior feedback. It incorporates